Reporte · Prueba técnica Python Developer de Integraciones

Punto 1 · Problemas de datos

Los encontré perfilando las respuestas de la API, cuadrando contra `X-Total-Count` y leyendo el código del mock (en un ERP real no lo tendría; perfilar y cuadrar sirve igual). Salvo que diga otra cosa, salieron del perfilado.

- **`status`** llega como "1", "0", "Activo", "Inactivo" o vacío → mapeo explícito; vacío o desconocido se rechaza (9): inventar un booleano es peor que no cargarlo.
- **`acquisition_cost`** llega como texto latino ("1.234,56"), número o vacío → "." son miles y "," el decimal; `Decimal` a 2 decimales, half-up. Vacío se rechaza (9).
- **`installed_at`** mezcla ISO con -05:00 y "13/04/2026" sin zona (18), día/mes porque 13 y 16 no son meses → asumo -05:00, como el resto de fechas del ERP, y paso a UTC con un aviso.
- **Nombres de 83 caracteres** (máximo 50) → trunco y lo registro en el log y en `warnings`.
- **`LOC-Z`** no existe en `/locations` → rechazo sin pedir lecturas (2), como dice el contrato.
- **La página 4 repite 3 activos** con un `updated_at` más nuevo (57 leídos, 54 únicos) → me quedo con el más reciente y lo registro.
- **Códigos con "/"** (EQ-012/A…): sus lecturas daban 404 → codifico el código en la URL; si no, eran rechazos falsos.
- **La página 3 responde 200 con un error en el cuerpo** y `data` vacía; tomándolo como fin se perdían 17 activos sin ninguna excepción → reintento, pagino hasta una página vacía y cuadro contra `X-Total-Count`.

Todo rechazo queda en `rejected_assets` con su motivo y el lote sigue; un activo ya cargado que llega inválido conserva su última versión válida.

Lo que sospecho que quedó sin encontrar:

- Las lecturas no se validan: un medidor que baja, un negativo o una fecha futura se cargan igual.
- Un activo borrado en el ERP sigue aquí, y un código con espacios u otras mayúsculas sería otro activo.

Punto 2 · Revisión de `transform_registry.py`

El nombre de la operación llega desde la web y termina en `eval`; ordené por gravedad y comprobé cada caso contra el original.

a. Qué puede salir mal, y cuándo se manifiesta:

- **Crítica · `eval` ejecuta cualquier código en el servidor:** basta un nombre con `(`, como `__import__('os').environ`.
- **Alta · el mismo `eval` puede tumbar el servicio:** basta `(9**9**9)`.
- **Alta, latente · `getattr` expone todo el módulo:** basta que alguien agregue un import o un helper de dos parámetros.
- **Media · las operaciones nuevas no tienen contrato:** una con otra firma o con un nombre repetido falla recién en producción.
- **Media · errores sin manejar:** una operación que ya no existe, `"1,5"` o un número en una de texto.
- **Media · `float` en montos:** `0.1` + `0.2` llega al ERP como `0.30000000000000004`; `"nan"` se acepta.
- **Baja · la compuesta devuelve número y la simple, texto:** cuando un cliente usa compuestas.
- **Baja · el CLI no valida argumentos, e `import os` sobra:** solo le facilita el ataque al `eval`.

b. Corrección mínima, sin cambiar las salidas legítimas: solo se puede llamar lo decorado con `@operation`, que valida la firma y rechaza nombres repetidos al importar (agregar una operación sigue siendo agregar una función). Las compuestas se mantienen, porque un cliente las usa, pero pasan por un evaluador con `ast` que solo acepta operaciones registradas. No renombré nada: los nombres son el contrato con la web. Supuesto: no sé qué compuestas hay en producción; lo revisaría en los logs antes de desplegar. Una operación desconocida lanza `OperationNotAllowed` en vez de `AttributeError`.

c. 35 tests (`docker compose run --rm --build tests` en `part2_transform_registry/`): mismas salidas que el original, 19 ataques rechazados sin ejecutarse y una operación nueva de otro archivo que funciona.

Además del código: hoy, avisar al líder (es un incidente de seguridad), buscar en los logs nombres con `(` sospechosos y, si aparecen, rotar credenciales. Después, coordinado: `Decimal`, coma decimal y tipos consistentes.

Punto 3 · Revisión de la propuesta de arquitectura

Reviso la propuesta como la de un compañero. Me gusta la dirección (eventos donde el ERP lo permita), pero hay piezas que no encajan con una base compartida y otras que se rompen. Entre paréntesis, el componente; diagrama en part3_architecture_review/diagram.md.

a. Qué no aplica a Fracttal

- **a1. Base de datos por cliente (2).** Mezcla dos modelos: una base por cliente choca con el contexto (hay una sola) y un esquema por cliente no escala aparte, porque vive en el mismo servidor. En mi experiencia las bases de los tenants cambian juntas: separarlas es aplicar cada cambio N veces.
- **a2. Un worker por cliente (3).** Se separa la CPU, pero la base sigue siendo una, y cada cliente nuevo es otro proceso que operar: el mismo problema de personal.

b. Qué está mal o se va a romper

- **b1. 200 solo al terminar (4).** Si procesar tarda más que el timeout del ERP (un pico, la base lenta), el ERP reenvía y el dato entra dos veces. Pediría 202 al guardarlo, y procesar después.
- **b2. Reintentar "hasta que responda" (6).** Si el ERP de un cliente cae un fin de semana o rechaza siempre un dato, se reintenta sin fin y lo demás espera; si la respuesta se pierde, el reintento duplica la orden. Pediría un máximo de intentos.
- **b3. Token en la memoria de cada proceso (7).** Si el ERP limita los tokens activos, las réplicas se invalidan entre sí (401 en cascada), sobre todo al desplegar. Pediría un almacén compartido (por ejemplo Redis) con una sola réplica renovando.

c. Qué falta antes de aprobarla

- **c1. Autenticación del webhook:** el cliente debe salir de la credencial, no del mensaje; si no, quien tenga la URL escribe en los datos de otra empresa.
- **c2. Idempotencia y orden:** que un reenvío no duplique y que un dato viejo no pise uno nuevo.
- **c3. ERPs que no pueden llamarnos** (on-premise o sin salida a internet): para ellos sigue el polling.
- **c4. Fallos:** cola de errores, alertas, métricas y límites por cliente; ¿cómo se entera el cliente de que algo suyo falló?
- **c5. Cifras:** clientes, eventos por segundo, tamaño de los payloads, qué es "tiempo real" y cuánto cuesta. Sin eso no sé si Kafka hace falta.
- **c6. Migración, para mí lo más delicado:** cómo conviven polling y eventos, en qué orden pasan los clientes y cómo se vuelve atrás.

d. Qué rescataría

- **d1. Eventos donde el ERP lo permita:** atacan la latencia y las consultas sin datos nuevos.
- **d2. Conectores por ERP con transformaciones en YAML (3 y 5),** si las diferencias entre clientes del mismo ERP (dos SAP con desarrollos propios) caben en configuración, empezando por los ERPs con más clientes. Reducen el código a medida, no lo eliminan.
- **d3. El bus (1):** desacoplar sí; Kafka según las cifras.
- **d4. La trazabilidad (8),** con los payloads fuera de la base compartida (S3 o equivalente) y retención.
- **d5. Un piloto o laboratorio con varios clientes simulados:** lo que se rompe (a2, b3, c4) aparece con muchos.

Uso de la IA

Usé Claude Code (modelo Claude Opus) en toda la prueba:

- **P1:** sondeó la API y leyó el mock (la primera lista de problemas fue suya; la comprobé con Postman y en la base); generó el ETL, `verify.py` y los tests bloque por bloque, cada uno después de que yo aprobara su plan, y probó AGENTS.md con dos agentes en sesión limpia.
- **P2, P3 y reporte:** encontró los problemas y escribió el hotfix y los tests (el alcance lo decidí yo); propuso críticas que discutí y reescribí con mi criterio; redactó el reporte con mis opiniones.

Qué generó mal y cómo lo detecté:

- Dijo que el token vivía 200 s; son 20 → leí el código del mock.
- Docker: un `compose.yaml` que no parseaba, una contraseña cambiada sin borrar el volumen y un `up` que corría el ETL antes de que existiera → al ejecutarlo.
- Sobreingeniería y decisiones sin consultarme: validaciones y archivos que no pedí, comentarios de más, inexactos (`literal_eval`) o en español → revisando el código; desde ahí, primero el plan y mi aprobación, después el código.
- Un nombre de proyecto Compose fijo: todas las copias compartían la base → la prueba en sesión limpia.
- En el P3 dijo que el polling incremental resolvía el problema 1 (el enunciado dice lo contrario), asumió un modelo multitenant y mezcló contexto de otros puntos → mis preguntas.

Qué no le delegué: las decisiones (Docker y Postgres fueron idea mía; las reglas de rechazo y la zona -05:00 me las propuso y las decidí yo; la estructura; las críticas del P3 que salen de mi experiencia), la verificación (las corridas, la base con DBeaver y psql, Postman, leer el código del ETL) y los commits. Solo usé las credenciales ficticias del mock.